

Reviewer Pack — Minimal Order of Kostant Cohomology and Circuit Monotone Wid...

Entry 23933cb261da · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 00:55:38 UTC

Minimal Order of Kostant Cohomology and Circuit Monotone Width Correlation

Entry ID: 23933cb261da **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 00:55:38 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (Kostant Cohomology) **Field B** (complexity object): Boolean Circuit Complexity

Statement:

For every n -vertex circuit C with depth d , the minimal order of the Kostant cohomology group $H^*(C)$ associated with C is $O(d^{2n/3})$. Equivalently: $\omega(C) = \Theta(O(d^{2n/3}))$ for a circuit C , where $\omega(C)$ denotes the monotone width of C .

Rationale (proposer's reasoning):

Kostant cohomology has been applied to quantum groups and their representations, which may provide insights into the complexity of circuits due to its ability to capture geometric properties of Lie algebras. This conjecture bridges this gap by suggesting a potential connection between the cohomological structure of circuits and their monotone width, a measure of circuit complexity.

Taxonomy category: REPRESENTATION_THEORY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b7615cd50b65335e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient (CC) between the minimal order of the Kostant cohomology group $H^*(C)$ and the monotone width $\omega(C)$ is within $0.8 \leq CC \leq 1$ for all circuits C with depth d , across at least 100 random seeds. The conjecture is falsified if any seed results in a correlation coefficient less than 0.8 or a metric mean of minimal orders greater than 10.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Kostant cohomology' AND 'monotone width' AND circuit - 'representation theory' AND 'Boolean circuit complexity' AND 'circuit depth' - cohomology group $H^*(C)$ AND 'circuit monotone width' AND $O(d^{(2n/3)})$

Top relevant hits considered: - [<http://arxiv.org/abs/0710.1443v2>] Variations on themes of Kostant - [<http://arxiv.org/abs/1907.12964v2>] Admissible restrictions of irreducible representations of reductive Lie groups: symplectic geometry and discrete decomp - [<http://arxiv.org/abs/2203.03476v1>] Monotone cohomologies and oriented matchings - [<http://arxiv.org/abs/1903.10609v3>] Circuit Complexity of Knot States in Chern-Simons theory - [<http://arxiv.org/abs/2512.15901v1>] Closed-Form Optimal Quantum Circuits for Single-Query Identification of Boolean Functions - [<http://arxiv.org/abs/2311.04204v3>] Sharp Thresholds Imply Circuit Lower Bounds: from random 2-SAT to Planted Clique

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n, d):
        # Simplified circuit generation (not actual Kostant cohomology)
        return [random.randint(0, 1) for _ in range(d * n)]

    def calculate_kostant_cohomology(circuit):
        # Placeholder function
        return len(set(circuit))

    def calculate_monotone_width(circuit):
        # Placeholder function
        return max(len(list(group)) for _, group in itertools.groupby(sorted(circuit)))

    n_values = [5, 10, 15, 20, 30, 40]
    results = []
    total_instances = 0
```

```

for n in n_values:
    for _ in range(10): # Ensure at least 30 instances per seed
        circuit = generate_circuit(n, random.randint(1, n))
        lcoh = calculate_kostant_cohomology(circuit)
        omega = calculate_monotone_width(circuit)
        results.append((lcoh, omega))
        total_instances += 1

if not results:
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

lcoh_values, omega_values = zip(*results)
mean_lcoh = sum(lcoh_values) / len(lcoh_values)
mean_omega = sum(omega_values) / len(omega_values)

correlation_coefficient = (sum((l - mean_lcoh) * (o - mean_omega) for l, o in results) /
                           math.sqrt(sum((l - mean_lcoh)**2 for l in lcoh_values) *
                                       sum((o - mean_omega)**2 for o in omega_values)))

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": total_instances,
    "n_max": max(n_values),
    "conjecture_holds": 0.8 <= correlation_coefficient <= 1 and mean_lcoh <= 10,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) /
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results if r["metric_value"] is not None) /
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient_out_of_range\" first_failing_seed={first_failing_seed}")
    else:

```

```
print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a9ff3bef.py", line 78, in <module>
    trial_result = run_trial(seed)
    ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a9ff3bef.py", line 59, in run_trial
    correlation_coefficient = (sum((l - mean_lcoh) * (o - mean_omega) for l, o in results) /
    ~~~~~
```

ZeroDivisionError: float division by zero

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a division by zero error before producing data that could confirm or falsify the conjecture. | next: Investigate and fix the ZeroDivisionError in the test code, then rerun the test with a valid dataset.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	24867
2	propose	ollama_remote	glm4:latest	0	0	32625
3	propose	ollama_remote	glm4:latest	0	0	13459
4	propose	ollama_remote	glm4:latest	0	0	47670
5	preregistration	ollama_remote	glm4:latest	0	0	38321
6	novelty	ollama_remote	glm4:latest	0	0	8613
7	novelty	ollama_remote	glm4:latest	0	0	35609
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10680
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14915
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15296
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14858
12	judge	ollama_remote	glm4:latest	0	0	26852

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 283766 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/23933cb261da.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/23933cb261da.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/23933cb261da.tar.gz` (if generated)